



**TECNOLÓGICO
NACIONAL DE MÉXICO**

Temas Selectos I

Mi Aldea

Ing. Gpe Khiabet Martínez Delgadillo

Docente: Dr. Mario Alberto Rodriguez Díaz

1. Introducción

El proyecto parte del código que ya teníamos de la práctica anterior: aquel mundo de arena con murciélagos y un gladiador. La estructura de clases estaba ahí y funcionaba, así que la idea fue aprovecharla para construir algo que no se le pareciera en nada.

Lo que se quería era un juego con alma de Animal Crossing, sin copiarlo. La premisa es sencilla: un pueblo con hierba, árboles y franjas de agua por donde caminas mientras te cruzas con aldeanos que se acercan a saludar, y objetos repartidos por el suelo que puedes recoger.

Las clases base (Sprite, TileSet, Tile) se quedaron tal cual, porque no había motivo para tocarlas. El trabajo se concentró en las clases que dependen de ellas, más dos clases nuevas que el juego de arena no tenía.

2. Descripción del videojuego

El juego se llama Mi Aldea. Es un mundo de 5000 × 5000 píxeles con una cámara que sigue al jugador, igual que antes. El cambio grande está en el ambiente: donde había una arena con columnas y murallas, ahora hay un pueblo con caminos de tierra, bosques, franjas de agua y parches de flores.

El jugador se mueve con las flechas, corre con Shift y recoge objetos del suelo con la barra espaciadora. Con la tecla E tala árboles, mina rocas e intercambia ítems con los aldeanos; con B construye casas o puentes, y N alterna entre esos dos modos de construcción. La tabla 1 resume los controles.

Tabla 1. Controles del juego

Tecla	Acción
Flechas	Mover personaje
Shift	Correr (velocidad ×2)
Espacio	Recoger ítem cercano
E	Talar árbol / Minar roca / Canjear con aldeano
B	Construir (casa o puente según el modo activo)
N	Cambiar modo de construcción (casa / puente)
X	Demoler construcción cercana
Esc	Salir del juego

El pueblo lo habitan seis NPCs: cinco aldeanos sencillos que pasean por su zona y se acercan a saludar cuando el jugador anda cerca, y un habitante especial, Toni-mon, que usa el algoritmo A* para llegar hasta el jugador esquivando obstáculos. La sección 3 entra en detalle sobre cada uno.

También hay un ciclo de día y noche, no es solo decorativo. Aunque visualmente pasa por varias etapas (amanecer, día, atardecer, crepúsculo y noche), para la lógica solo cuentan dos fases, «día» y «noche», y cada cambio de fase se altera el juego: cambia los ítems que aparecen, las ofertas de los aldeanos, reinicia el contador de intercambios y activa un objetivo nuevo.

2.1 El mapa

El mapa maneja ocho tipos de terreno. La hierba verde cubre casi todo el mundo y es el terreno libre por excelencia. Los caminos de tierra forman una cruz que cruza el centro; se ven más oscuros que la hierba y A* los trata como terreno lento, con costo doble, de modo que los NPCs que usan A* tienden a rodearlos cuando pueden.

Los árboles se agrupan en bosques, sobre todo en las esquinas y los bordes, y son obstáculos sólidos: nadie los atraviesa, ni el jugador ni los aldeanos. El agua no forma un lago en una esquina, sino una cruz que recorre el mapa de lado a lado y de arriba abajo, cruzándose en el centro; también bloquea el paso, salvo donde se construya un puente. Las flores, en cambio, están dispersas y son solo decorativas; no estorban y muchas veces hay ítems cerca de ellas.

Pese al tamaño del mundo, el juego carga rápido. El truco es que el mapa completo se pregenera una sola vez en una superficie, y durante la partida solo se hace un blit de esa imagen ya lista. Los sprites de árboles y rocas (cargados de arboles.png, piedras.png y piedritas.png) se dibujan encima en cada frame a partir del diccionario sprites_mundo, que guarda la posición y el nombre del sprite de cada objeto: uno por árbol o roca, sin duplicados.

Tabla 2. Tipos de tile del mapa

Tipo	Nombre	Descripción	Bloquea paso
0	Hierba	Terreno base, libre	No
1	Camino	Cruz central, costo $\times 2$ en A*	No
2	Árbol	Bosques en esquinas	Sí
3	Agua	Franja en cruz — cruzable con puente	Sí (sin puente)
4	Flores	Decorativo, pisable	No
5	Roca grande	Minable, da 5–8 de piedra	No
6	Piedrita	Minable, da 1–2 de piedra	No



Imagen 1. Vista segmentada de Mi Aldea

2.2 Sistema de intercambio y objetivos diarios

Al arrancar la partida se siembran 35 objetos repartidos al azar por el mapa. Solo caen en terreno libre (hierba, caminos o flores), nunca dentro de un obstáculo ni en el agua. Los tipos que existen son:

- Manzana: un círculo rojo con tallo verde. Es el objeto más común.
- Naranja: un círculo naranja con brillo, igual que la manzana pero de otro color.
- Flor: cinco pétalos rosas con un centro amarillo claro.
- Concha: una elipse color crema con líneas de arco que imitan la textura de una concha de mar.
- Estrella de mar: un polígono de diez puntas en amarillo.

Todos comparten la misma animación de brillo: un halo blanco semitransparente que aparece y se desvanece despacio, para que sean fáciles de ubicar. Cuando el jugador recoge un ítem, este desaparece para siempre; en esta versión no hay reaparición, así que el mapa se va vaciando a medida que juegas.

Tabla 3. Ítems de valor por fase

Ítem de valor	Fase	Cómo obtenerlo
sol_cristal	Día	Canjear con aldeano (2 flores o 2 conchas)
flor_dorada	Día	Canjear con aldeano (3 manzanas o 1 estrella)
luna_piedra	Noche	Canjear con aldeano (2 flores o 2 naranjas)
estrella_fugaz	Noche	Canjear con aldeano (2 conchas o 1 estrella)

Los objetivos van por fase: tres distintos para el día y tres para la noche. Cada uno pide ítems concretos, construcciones y/o cierto número de intercambios con aldeanos. Si no consigues los ítems de valor que corresponden a la fase activa, el objetivo no se puede cerrar. Al completarlo recibes un bono de recursos y se pasa al siguiente.

2.3 Sistema de construcción

Talar árboles (E junto a uno) da madera. Minar una roca grande del tipo 5 deja entre 5 y 8 piedras; una piedrita del tipo 6 apenas da 1 o 2. Con esos recursos el jugador construye lo que muestra la tabla 4.

Tabla 4. Recetas de construcción

Construcción	Madera	Piedra	Efecto
Casa	5	3	Cuenta para el objetivo del día
Puente	4	0	Abre un cruce en el agua (quita los obstáculos)

Las construcciones usan sprites reales, *casa.png* y *puente.png*, y si esos archivos faltan el juego dibuja versiones procedurales de respaldo. Hay una diferencia en dónde se puede colocar cada una: la casa se levanta junto al jugador sin más, pero el puente solo se construye si hay agua cerca; si intentas ponerlo en seco, el juego te avisa que te acerques al agua. Al construirlo, elimina los obstáculos de agua a su alrededor y abre un tramo transitable sobre la franja, que es la única forma de cruzar al otro lado.

También se puede demoler con la tecla X. El juego busca la construcción más cercana dentro de un radio, te devuelve la mitad de los materiales y la quita del mapa. Si lo que demueles es un puente, el agua vuelve a cerrarse en ese punto y el cruce desaparece, así que conviene pensarlo dos veces antes de quitar uno.

2.4 Ciclo de día y noche

El ciclo dura 6000 frames. El color del océano (el fondo del mapa, que da el efecto de isla) y el overlay del agua transitan entre tonos de azul: del azul marino nocturno (8, 28, 72) al azul claro diurno (50, 140, 210) y de vuelta. Visualmente el día pasa por amanecer, día pleno, atardecer y crepúsculo antes de volver a la noche, pero para la lógica del juego solo importan dos fases: «día» y «noche».

Cada vez que se cambia de fase ocurren varias cosas a la vez: aparecen ítems de valor nuevos acordes a la fase, los aldeanos actualizan sus ofertas de intercambio, se reinicia el contador de trades y se activa un objetivo nuevo. Por eso el ciclo no es un simple adorno: marca el ritmo de lo que puedes conseguir en cada momento.

Hay además un pequeño detalle que ayuda a que el mundo no se agote: cada 400 frames el juego hace crecer un árbol nuevo en una posición libre al azar, de modo que siempre queda algo de madera por talar aunque lleves rato jugando.

3. Habitantes

El pueblo no estaría vivo sin quien lo camine. En Mi Aldea hay tres clases de habitantes, y aunque a primera vista los tres se mueven y animan parecido, cada uno responde a una lógica distinta. Esta sección los describe uno por uno: qué son, cómo deciden a dónde ir (su máquina de estados) y cómo se animan.

3.1 El personaje (jugador)

Descripción

El personaje es el sprite *monitoB.png*, un monito animado que camina en cuatro direcciones y se controla con las flechas. Si mantienes Shift corre y dobla su velocidad. La cámara siempre lo deja centrado en pantalla, igual que en la práctica anterior.

Lo nuevo de esta entrega es el inventario. El personaje carga hasta 12 objetos: te acercas a un ítem, presionas espacio, el ítem desaparece del mapa y aparece en la barra inferior. Al recogerlo también sale un aviso breve en el centro de la pantalla con el nombre del objeto. Un detalle: si un ítem de valor no corresponde a la fase activa (de día o de noche), el personaje no lo recoge.

Máquina de estados

El personaje tiene tres estados internos que deciden qué animación se reproduce. QUIETO cuando la velocidad es cero; CAMINANDO cuando se mueve sin Shift; y CORRIENDO cuando Shift está activo. El cambio entre ellos es inmediato y depende solo de la entrada del teclado, sin temporizadores de por medio.

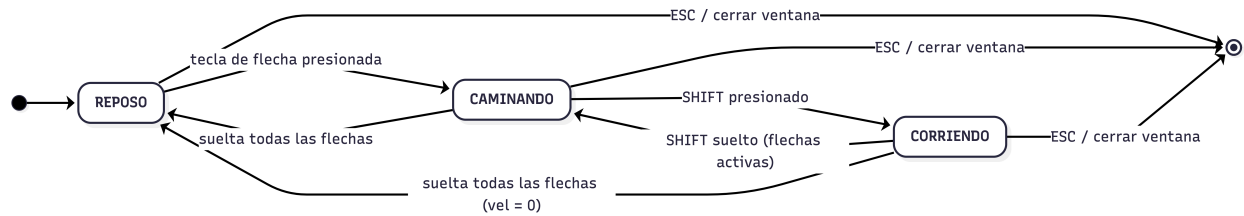


Figura 3.1. Máquina de estados — clase Personaje

Animaciones

Cada estado tiene su propio juego de fotogramas por dirección, recortados del sprite *monitoB.png*. En QUIETO se muestra el primer fotograma de la última dirección mirada; en CAMINANDO y CORRIENDO se recorre el ciclo de pasos, solo que correr avanza los fotogramas más rápido para que la animación acompañe la velocidad mayor.



Imagen 3.2. Animación del personaje en sus cuatro direcciones

3.2 Los aldeanos

Descripción

Son cinco y andan repartidos por distintas zonas del mapa: Pico, Luna, Mochi, Dali y Benji. Cada uno lleva un color propio para distinguirlos de un vistazo. No usan ninguna imagen: se dibujan por completo con código y formas geométricas (cabeza redonda, orejas triangulares de animalito, cuerpo ovalado y piernas que se mueven al caminar).

Máquina de estados

El comportamiento descansa en una máquina de tres estados. En DEAMBULAR, que es el estado por defecto, el aldeano elige un punto al azar dentro de su zona, camina hasta él, espera unos segundos y vuelve a escoger otro destino. Es lógica simple, pero basta para que parezcan tener vida propia.

Cuando el jugador se acerca a menos de 220 píxeles, el aldeano pasa a SALUDAR: avanza despacio hacia él y muestra una burbuja con su oferta de intercambio (por ejemplo, cuántas flores pide y qué entrega a cambio). Pasado un momento, o si el jugador se aleja más de 380 píxeles, cambia a REGRESAR y vuelve caminando a su punto de origen, donde retoma DEAMBULAR.

La limitación que conviene tener presente es que los aldeanos simples no esquivan obstáculos. Si un árbol se les atraviesa, se detienen y esperan a que el temporizador genere un destino nuevo. Esto se nota cerca de los bosques, donde a veces quedan un instante «pegados» contra un árbol antes de reencaminarse.

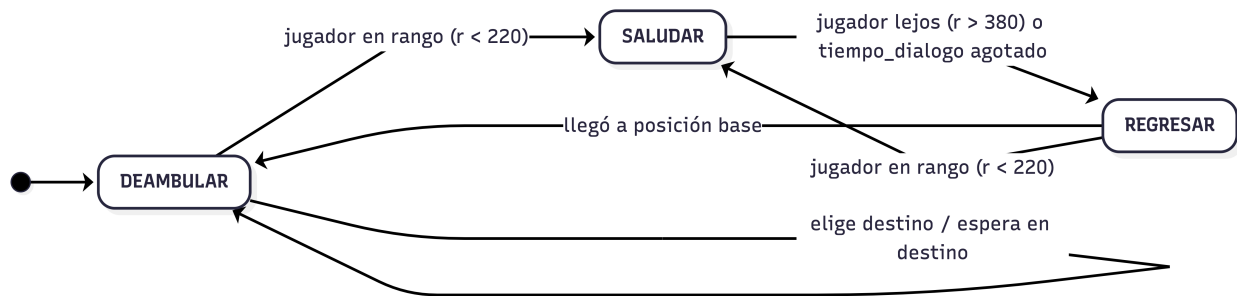


Figura 3.3. Máquina de estados — clase Aldeano

Animaciones

Al dibujarse por código, su animación no viene de un sprite sino de las propias formas. Las piernas oscilan mientras el aldeano se desplaza y quedan fijas cuando está parado, así que el simple hecho de moverse ya basta para que se vea la caminata. El color del cuerpo es lo único que cambia de un aldeano a otro.



Imagen 3.4. Aldeanos dibujados por código, cada uno con su color

3.3 Habitante especial: Toni - mon

Descripción

Toni es el sexto NPC y, a la vista, es idéntico a los demás aldeanos: usa el mismo dibujo procedural. La diferencia está en cómo se mueve. Tiene su zona en la franja centro este del mapa y, cuando detecta al jugador, calcula una ruta con A*, tal como hacía el Gladiador del proyecto anterior. Sus radios de detección no son fijos: empieza a saludar cuando el jugador entra en unos 220 píxeles, retoma el saludo desde 260 si venía de vuelta, y solo lo suelta cuando el jugador se aleja más de 450.



Imagen 3.4.1. Aldeano Toni-mon

Máquina de estados

Comparte la misma estructura de tres estados que el aldeano común (DEAMBULAR, SALUDAR y REGRESAR), pero por dentro SALUDAR y REGRESAR se apoyan en A*. La diferencia se nota enseguida en el movimiento: mientras los aldeanos normales van en línea recta y se traban contra los obstáculos, Toni siempre encuentra el camino aunque tenga que rodear un bosque entero o bordear una franja de agua. La ruta se recalcula cada 20 frames por si el jugador se desplaza, y también usa A* para volver a su base cuando el jugador se aleja. En el HUD de depuración se puede ver en qué estado está en cada momento: DEAMBULANDO, SALUDANDO o REGRESANDO.

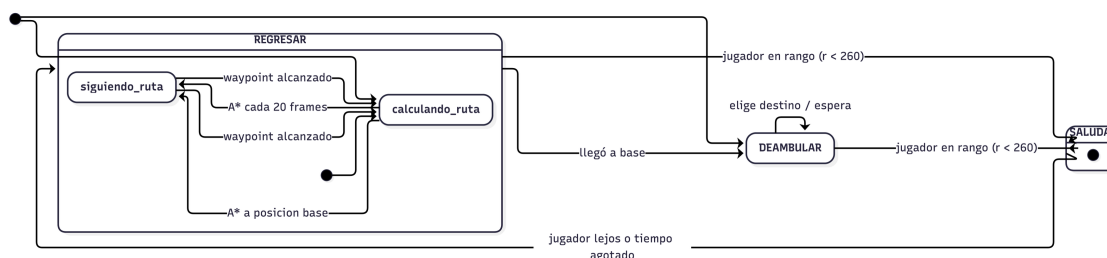


Figura 3.5. Máquina de estados — clase Habitante (con A* en SALUDAR y REGRESAR)

Animaciones

Como hereda del aldeano, Toni se anima exactamente igual: las piernas se mueven al caminar y se quedan quietas al detenerse. Lo que cambia no es la animación sino la trayectoria; al rodear obstáculos, el monito traza curvas en lugar de líneas rectas, y eso por sí solo lo hace ver más «consciente» del terreno que el resto.

3.4 Aldeano frente a Habitante

Los aldeanos y Toni comparten apariencia y máquina de estados; toda la diferencia está en cómo se mueven. El aldeano va en línea recta y se traba ante cualquier obstáculo, mientras que el habitante recalcula su ruta con A* y los rodea. Es un buen ejemplo de herencia bien usada: se reutiliza todo el comportamiento del padre y solo se sobrescriben los dos métodos que cambian (`_saludar` y `_regresar`). La tabla 5 resume el contraste.



Imagen 3.4.2. Aldeano parlante

Tabla 5. Comparativa entre Aldeano y Habitante

Aspecto	Aldeano (Pico, Luna, Mochi, Dali, Benji)	Habitante (Toni)
Herencia	Clase base, no hereda de nadie	Hereda de Aldeano
Cantidad	5 instancias	1 instancia
Dibujo	Procedural en <code>_gen_frames()</code>	El mismo (heredado)
Movimiento	En línea recta hacia el destino	Ruta con A* (<code>_astar</code>)
Ante obstáculos	Se traba y espera otro destino	Los rodea recalculando la ruta
Conoce el mapa	No	Sí, recibe <code>mapa_celdas</code>
Recálculo de ruta	No aplica	Cada 20 frames
Radio de visión	220 píxeles	220 / 260 según el estado
Estados	DEAMBULAR, SALUDAR, REGRESAR	Los mismos (sobrescribe 2)

4. Imágenes y recursos

4.1 Imágenes reutilizadas del proyecto anterior

Se reaprovecharon varios archivos que ya teníamos:

- `monitoB.png`: el sprite del personaje principal. No se tocó; funciona igual que antes.
- `Arena_Tileset.png`: base para el suelo del pueblo. Las filas 8 y 9 dan los tiles de pasto que usamos como hierba, y las filas 13 a 16 dan los caminos de tierra.
- `arboles.png`: franja con 4 árboles pixel art (968×432, fondo negro). Se cargan y recortan al inicio.
- `piedras.png`: 3 variantes de roca grande (478×188, fondo negro).
- `piedritas.png`: 7 variantes de piedrita pequeña (622×164, fondo negro).
- `casa.png`: sprite de casa pixel art (314×306, fondo negro), usado al construir una casa.
- `punte.png`: sprite de puente (112×120, fondo negro), usado al construir un puente.

4.2 Gráficos generados por código

Los aldeanos, los árboles, el agua, las flores y los objetos recogibles se dibujan directamente con funciones de `pygame` (`pygame.draw.circle`, `polygone`, `rect`, `ellipse`). No dependen de ningún archivo de imagen, lo cual nos quitó de encima el problema de buscar sprites y dejó el juego corriendo en cualquier máquina sin recursos extra.

5. Estructura del código

El proyecto se reparte en dos archivos, igual que antes: *Sprites_pueblo.py* con todas las clases y *mundo_pueblo.py* con el bucle principal del juego.

5.1 Clases sin cambios

Estas se copiaron directamente del proyecto anterior porque no había razón para tocarlas:

- **Sprite**: la clase base que carga la imagen y define el rectángulo de colisión. Solo se le añadió un `if` para que no falle si el archivo de imagen no existe.
- **TileSet**: hereda de **Sprite**, carga el `tileset` y extrae los frames individuales.
- **Tile**: representa una celda del mapa con su posición, tipo e imagen. Igual que antes.
- `_astar()`: sin cambios. Soporta tiles de costo variable (tipo 1 = costo $\times 2$).

5.2 TileSetPueblo

Hereda de **TileSet**, como **TileSetArena**, pero genera un mapa de aldea en lugar de una arena. La mecánica interna es la misma: se arma una cuadrícula, se asigna un tipo a cada celda y se dibujan los tiles encima. Los tipos son

0 = hierba (libre),

1 = camino de tierra (costo $\times 2$ en A^*),

2 = árbol (obstáculo),

3 = agua (obstáculo) y

4 = flores (decorativo, pisable).

El mapa se arma en pasos: primero se rellena todo de hierba, luego se traza la cruz de caminos, después se colocan grupos de árboles en distintas zonas, se generan las franjas de agua en forma de cruz que cruzan el mapa, se reparten rocas y piedritas, y por último se esparcen parches de flores al azar.

5.3 SpritesExterior

Carga *arboles.png*, *piedras.png*, *piedritas.png*, *casa.png* y *puente.png* al inicio. Vuelve transparente el fondo negro con `set_colorkey((0,0,0))`, sin recorrer píxel por píxel. Recorta cada sprite según coordenadas predefinidas y los guarda en `self.sprites`, y los escala manteniendo proporción con `get(nombre, tam_px)`.

5.4 Aldeano

Es el equivalente al Murciélago del proyecto anterior, pero amigable. Comparte la misma máquina de tres estados: **DEAMBULAR**, **SALUDAR** y **REGRESAR**. No hereda de **TileSet** porque se dibuja con código y no necesita cargar ninguna imagen.

5.5 Habitante

Hereda de **Aldeano**, así como el **Gladiador** heredaba del Murciélago. Lo distinto es que sus estados **SALUDAR** y **REGRESAR** usan A^* para moverse, de modo que rodea obstáculos

correctamente. Recibe `mapa_celdas` como parámetro para que A* conozca el costo de cada tipo de tile.

5.6 Construcción

Guarda el tipo (casa o puente), la posición y una referencia al gestor de sprites. Al dibujarse usa el PNG si está disponible y, si no, recurre al dibujo procedural de respaldo. Las recetas son: casa = 5 madera + 3 piedra y puente = 4 madera.

La lógica de colocar un puente solo cerca del agua, de abrir el cruce al construirlo y de cerrarlo de nuevo al demolerlo vive en el bucle principal (funciones `intentar_construccion` e `intentar_demolicion`), que se apoya en los métodos `abrir_paso_rio` y `cerrar_paso_rio` del `TileSetPueblo`.

5.7 Objetivo

Define los objetivos diarios por fase. Cada objetivo tiene `items_req` (diccionario de ítems necesarios), `const_req` (construcciones necesarias) y `trades_req` (intercambios mínimos). El método `verificar()` revisa los tres requisitos y devuelve `False` si falta alguno; al completarse, `siguiente()` activa el siguiente objetivo del ciclo.

5.8 Ítem (clase nueva)

Completamente nueva. Un Ítem es un objeto recogible (manzana, naranja, flor, concha o estrella de mar) con animación de brillo. El jugador lo levanta con espacio cuando está cerca, y cada tipo se dibuja con una forma distinta usando `pygame.draw`.

5.9 Personaje

Es básicamente el mismo de antes, con tres añadidos: una lista inventario (máximo 12 objetos), la variable `recoger_activo` que se activa al presionar espacio, y el método `recoger_item()`, que comprueba si hay algún ítem cerca y lo guarda en el inventario.

6. Diagramas

Esta sección reúne los diagramas que describen la estructura y el comportamiento del programa. El diagrama de flujo muestra cómo corre el juego y el diagrama de clases, cómo se relacionan entre sí. Las máquinas de estado de cada habitante ya quedaron en la sección 3, junto a su descripción.

6.1 Diagrama de flujo general

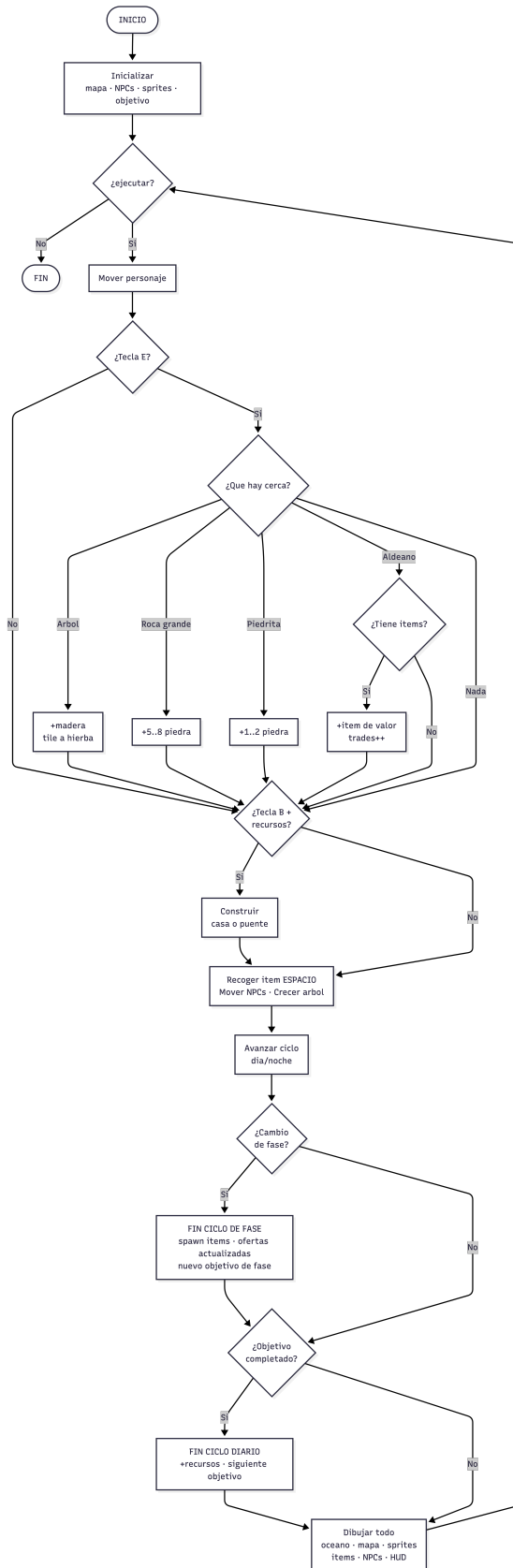


Figura 6.1. Diagrama de flujo general del bucle principal del juego

6.2 Diagrama de clases

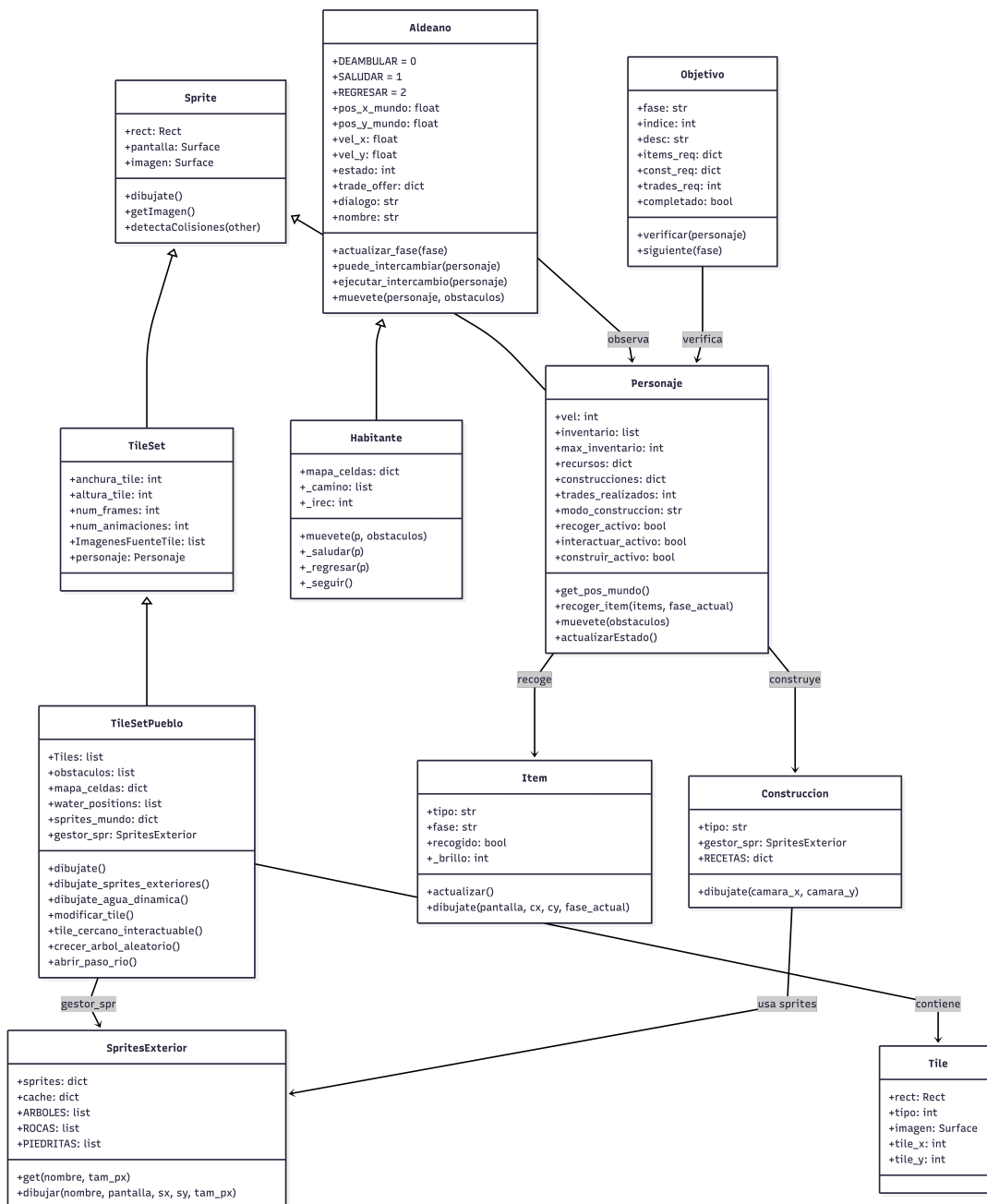


Figura 6.2. Diagrama de clases

6.3 Máquina de estados (clase Item)

La única máquina de estados que no corresponde a un habitante es la del Item, que regula su ciclo de brillo y su recogida.

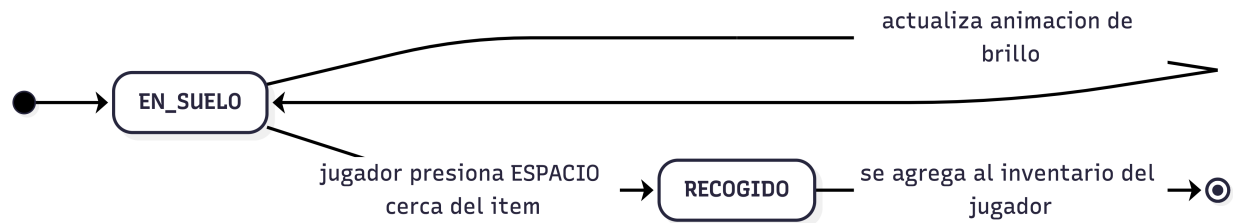


Figura 6.3. Máquina de estados — clase Item

7. Conclusión

Este proyecto me dejó claro para qué sirve la herencia en la programación orientada a objetos: para reutilizar comportamiento que ya funcionaba sin reescribirlo. El ciclo de día, los aldeanos y el inventario son cosas que antes no existían, pero el movimiento del personaje, A* y la cámara funcionan exactamente igual que en el mundo de arena.

El contraste entre Aldeano y Habitante es lo que mejor resume la idea: visualmente hacen lo mismo, pero uno llega a su destino en línea recta y se traba, mientras el otro rodea cualquier obstáculo. Misma apariencia, decisión distinta.

Como siguiente paso se podría añadir música de fondo y que los aldeanos digan frases distintas según la hora. También estaría bien que los ítems reaparecieran con el tiempo, porque tal como está ahora el mapa se va quedando vacío conforme juegas.